



AMERICAN INTERNATIONAL UNIVERSITY – BANGLADESH (AIUB)

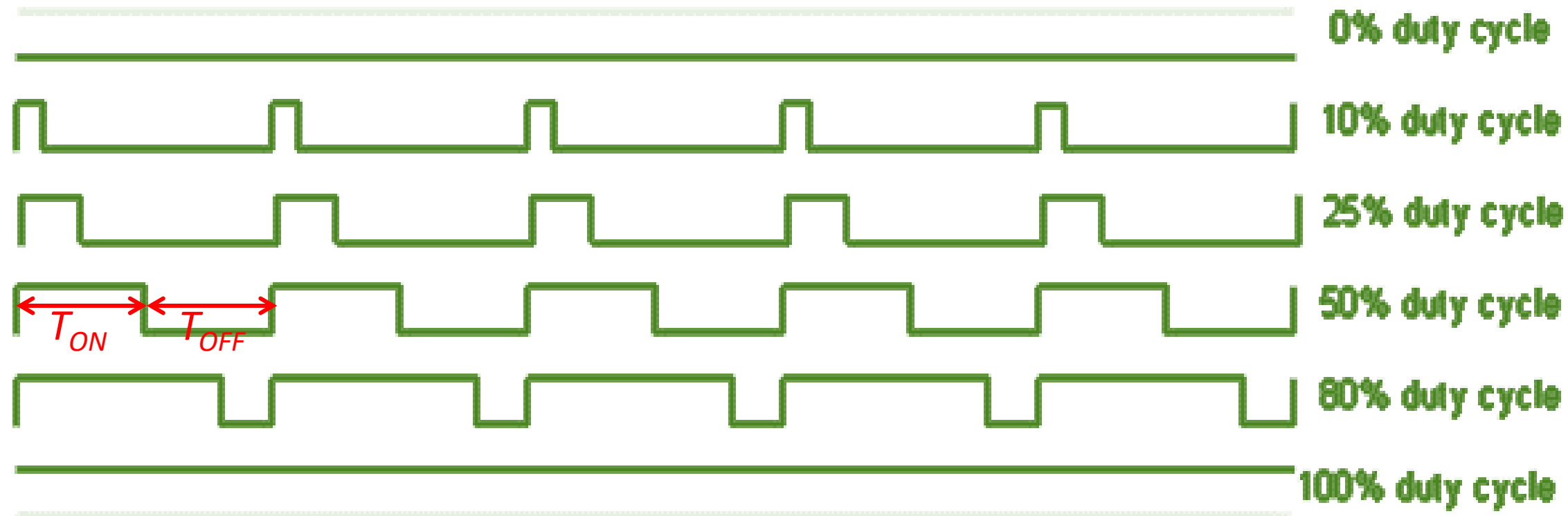
Where leaders are created

Lecture # 2 (Final) Pulse Width Modulation (PWM)

EEE Department, AIUB

PWM

Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off. This on-off pattern can simulate voltages in between the full V_{CC} of the board (e.g., 5 V on UNO) and off (0 Volts). The duration of "on time" is called the **pulse width**. To get varying analog values, the pulse width can be varied. If this on-off pattern is repeated fast enough with an LED, for example, the result is as if the signal is a steady voltage between 0 and V_{CC} controlling the brightness of the LED.



The duty cycle is defined as the ratio of ON pulse duration to the time period.

$$D = \frac{T_{ON}}{T} \times 100\%$$

Here, T_{ON} = ON pulse duration

T = Time period = $T_{ON} + T_{OFF}$,

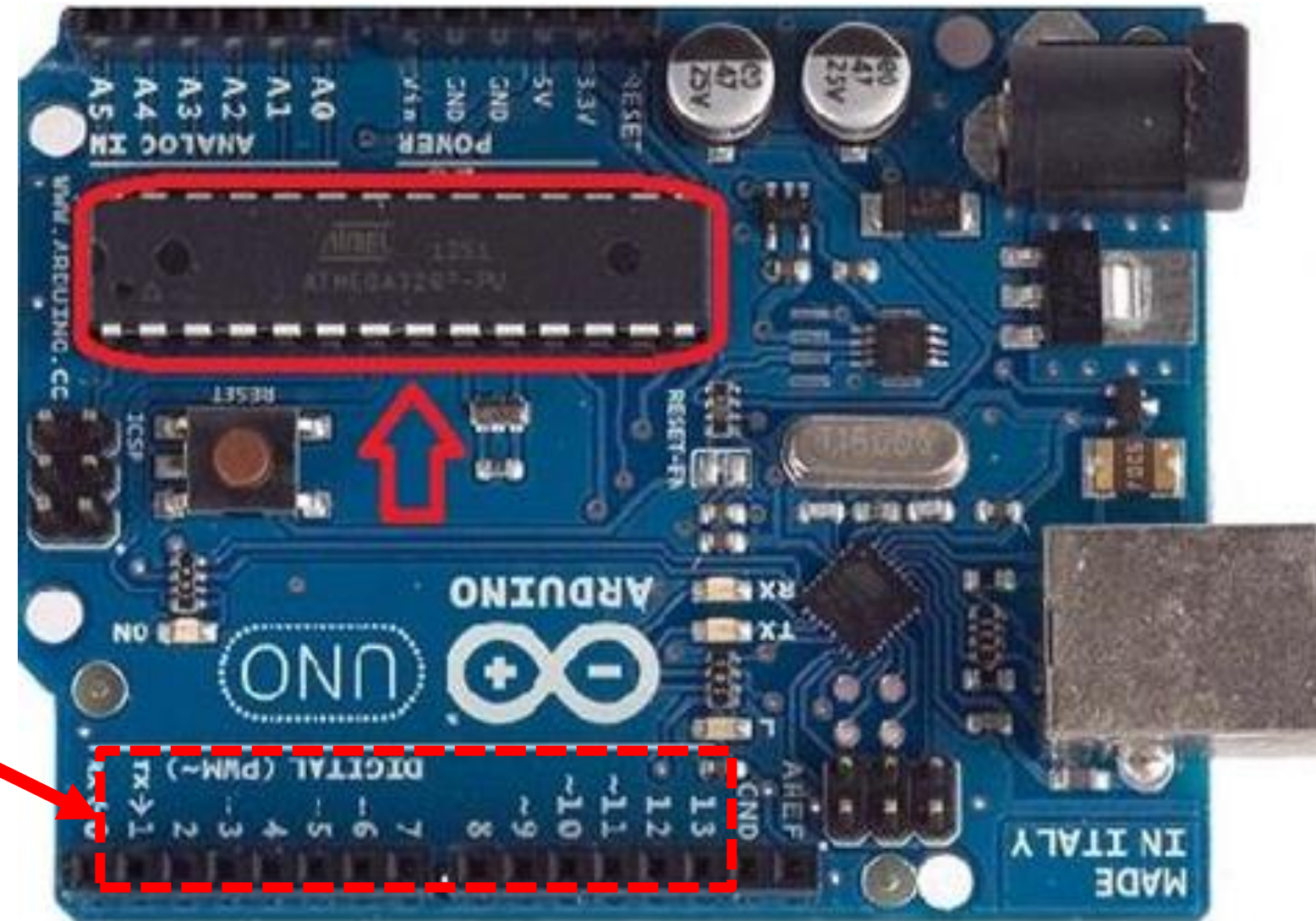
where T_{OFF} = OFF pulse duration

PWM

The Arduino's programming language makes PWM easy to use; simply call the built-in function **`analogWrite(pin, dutyCycle)`**, where **dutyCycle** is a value **from 0 to 255**, and **pin** is one of the PWM pins (**3, 5, 6, 9, 10, or 11**).

The `analogWrite()` function provides a simple interface to the hardware PWM, but doesn't provide any control over frequency.

Note the tilde ~ sign with the pin numbers of the image.



PWM

The **main PWM modes** are "Fast PWM" and "Phase-correct PWM".

There are **two Fast PWM modes** for Timer/Counter 0 unit. These are **modes 3 and 7**, which are selected using the waveform generation mode bits (WGM02, WGM01, and WGM00). The WGM01 and WGM00 bits are located in the TCCR0A register. The WGM02 bit is located in the TCCR0B register.

There are **2 types** of Fast PWM modes: (a) Non-inverting and (b) Inverting Fast PWM

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x27 (0x47)	OCR0A[7:0]								OCR0A
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x28 (0x48)	OCR0B[7:0]								OCR0B
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x26 (0x46)	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk _{I/O} /(No prescaling)
0	1	0	clk _{I/O} /8 (From prescaler)
0	1	1	clk _{I/O} /64 (From prescaler)
1	0	0	clk _{I/O} /256 (From prescaler)
1	0	1	clk _{I/O} /1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

PWM

The difference between the mode 3 and mode 7 Fast PWMs is their TOP values. For mode 3, the TOP value is fixed (0xFF), whereas that for mode 7 is OCRnA value. So, if mode 7 is used then we have to **load a count value** into the **OCRnA register**. Once a mode is selected, the timer/counter starts counting from BOTTOM to TOP value and when the TOP value is reached the counting is repeated from BOTTOM.

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	—	—	—
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	—	—	—
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

The table on the left shows the **WGM bits combination** for modes 3 and 7.

Fast PWM Mode: Setting Modes

In the fast PWM mode, the Compare Output Mode **bits COM0A1, COM0A0 (in the TCCR0A register)** for the output at OC0A and **bits COM0B1, COM0B0 (in the TCCR0A register)** for the output at OC0B, are used to configure the output as either non-inverting or inverting mode of operation. **Table 12-3** shows how to select COM0A1, COM0A0 bits to generate non-inverting and inverting mode's fast PWM waveform.

Table 12-3. Compare Output Mode, Fast PWM Mode⁽¹⁾

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	<u>Clear OC0A on Compare Match, set OC0A at BOTTOM, (non-inverting mode).</u>
1	1	<u>Set OC0A on Compare Match, clear OC0A at BOTTOM, (inverting mode).</u>

Fast PWM Mode: Setting Modes

In the fast PWM mode, the compare unit allows the generation of PWM waveforms on the OC0x pins. [Table 12.6](#) shows the COM0B1:0 bit functionality when the WGM02:0 bits are set to the fast PWM mode.

Table 12-6. Compare Output Mode, Fast PWM Mode⁽¹⁾

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	Clear OC0B on Compare Match, set OC0B at BOTTOM, <u>(non-inverting mode)</u>
1	1	Set OC0B on Compare Match, clear OC0B at BOTTOM, <u>(inverting mode).</u>

Note: 1. A special case occurs when OCR0B equals TOP and COM0B1 is set. In this case, the Compare Match is ignored, but the set or clear is done at TOP. See ["Fast PWM Mode" on page 101](#) for more details.

Fast PWM Mode: Setting Modes

Setting the COM0x1:0 bits to **10** will produce a **non-inverted** PWM and an **inverted** PWM output can be generated by setting the COM0x1:0 to **11**.

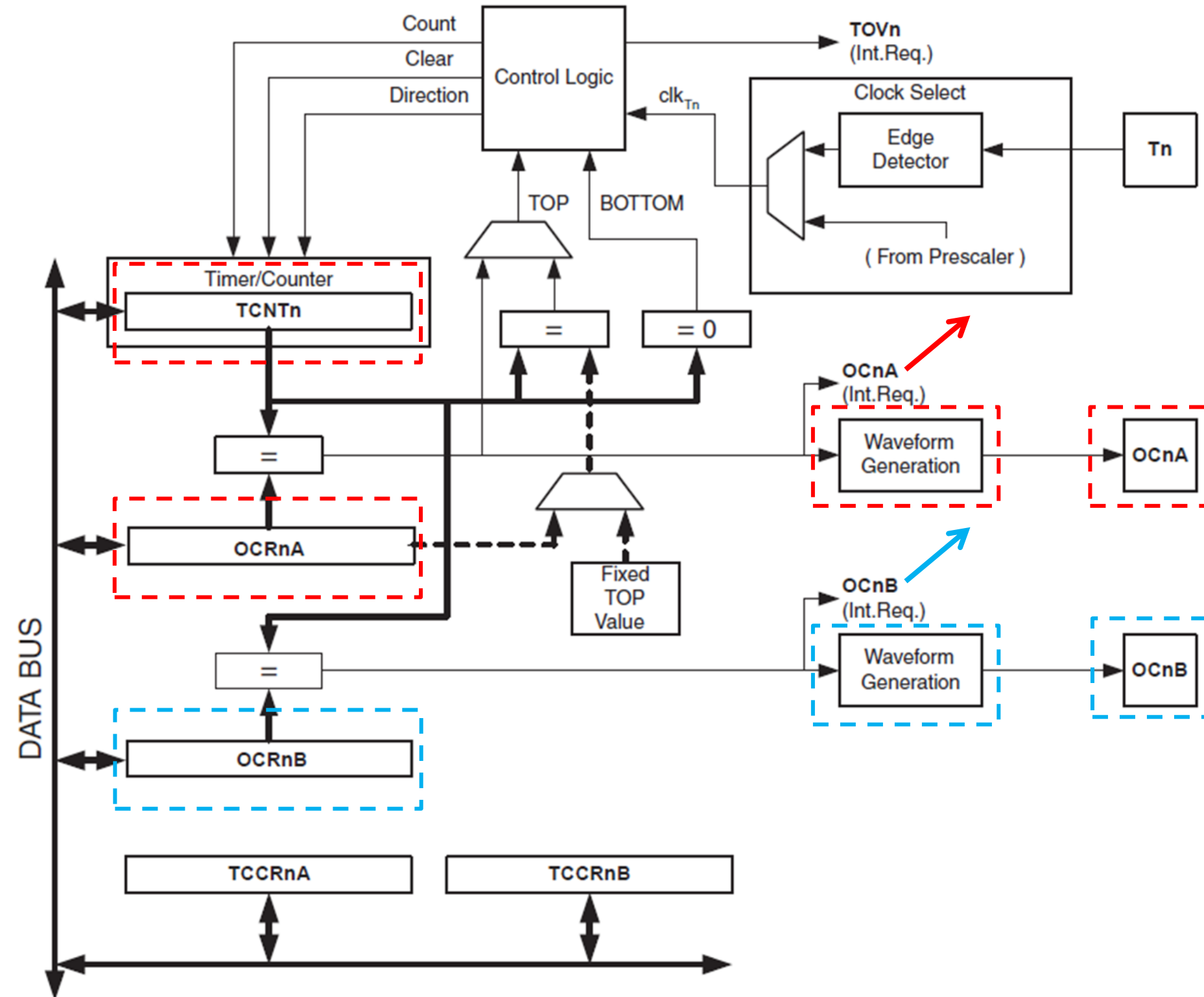
Setting the COM0A1:0 bits to **01** allows the OC0A pin to **toggle** on Compare Matches if the **WGM02 bit is set**. This option is not available for the OC0B pin (see [Table 12-6](#)).

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Fast PWM Mode

- ❑ The **Fast Pulse Width Modulation** or **Fast PWM Mode** ($WGM02:0 = 3$ or 7) provides a high-frequency PWM waveform generation option. The **Fast PWM** differs from the other PWM option in its **single-slope operation**.
- ❑ The counter starts counting from **BOTTOM** to **TOP** and then restarts from **BOTTOM**.
- ❑ In **Non-inverting Compare Output Mode**, the Output Compare (OC0x) is **cleared** on the **compare match** between **TCNT0** and **OCR0x**, and **set** at **BOTTOM**.
- ❑ In **Inverting Compare Output Mode**, the output is **set** on compare match and **cleared** at **BOTTOM**.



Fast PWM Mode

- ❑ Due to the single-slope operation, the operating frequency of the **Fast PWM mode** can be **twice as high as** the phase correct PWM mode that uses dual-slope operation.
- ❑ This high frequency makes the fast PWM mode well suited for **power regulation, rectification, and DAC applications**.
- ❑ High frequency allows physically small-sized external components (coils, capacitors), and therefore **reduces** the total system cost.
- ❑ In fast PWM mode, the counter is incremented until the counter value **matches the TOP** value, the register TCNTn counts from bottom value to maximum value stored in the register OCRn. The counter is then cleared/reset to zero at the following timer clock cycle. If the timer is configured in non-inverting mode, **PWM output pin (OCn)** goes low when the value of the above two registers matches. The OCn pin becomes high when the **TCNTn register reaches at bottom value**. In inverting mode, **OCn pin behaves opposite to non-inverting mode**.

Fast PWM Mode

- The timing diagram for the **Fast PWM Mode** is shown in Figure 12.6.
- The TCNT0 value is in the timing diagram shown as a histogram for illustrating the **single-slope operation**.
- The diagram includes **non-inverted and inverted PWM outputs**.

The **small horizontal line marks** on the TCNT0 slopes represent **compare matches** between OCR0x and TCNT0.

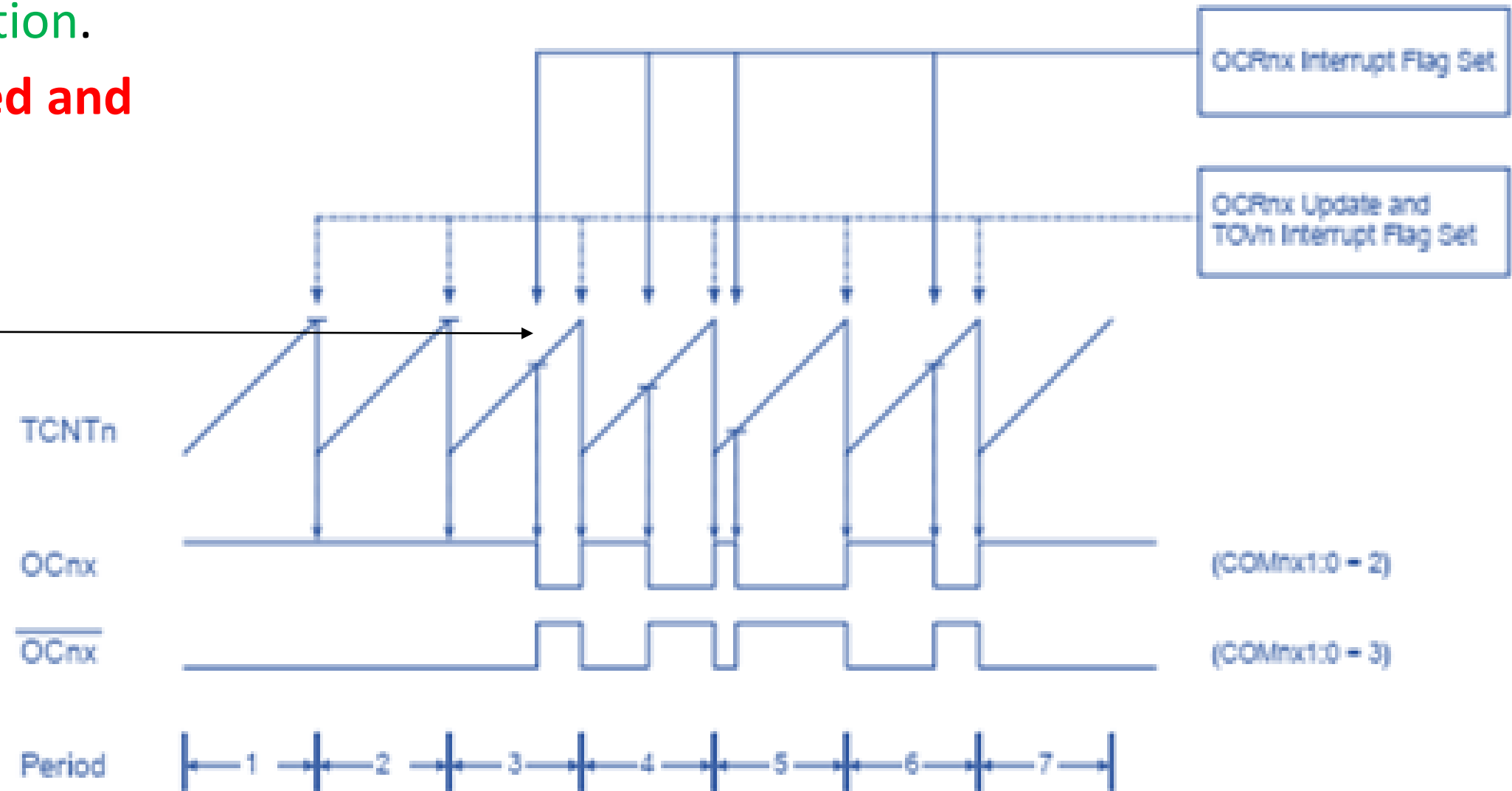


Fig. 12.6 Timing Diagram of Fast PWM Mode

Fast PWM Mode: Output Frequency

- The actual **OC0x** value will only be visible on the port pin if the data direction for the port pin is set as **output**.
- The PWM waveform is generated by **setting (or clearing) the OC0x Register** at the **compare match between OCR0x and TCNT0**, and clearing (or setting) the OC0x Register at the timer clock cycle the counter is cleared (i.e., from TOP to BOTTOM).
- The PWM frequency for the output can be calculated by the following equation:

$$f_{Fast_PWM} = \frac{f_{clk_IO}}{N \times (TOP + 1)}$$

For Fast PWM of Mode 3, $TOP = 255$
For Fast PWM of Mode 7, $TOP = OCRA$

The N variable represents the pre-scale factor (1, 8, 64, 256, or 1024).

Fast PWM Mode: Output Frequency

- **Non-Inverting Fast PWM Duty Cycle**

The duty cycle of the **Non-Inverting mode** Fast PWM signal is calculated using the following formula.

$$D = \frac{OCR0x}{(TOP+1)} \cdot 100\% ; x = A \text{ or } B$$

For Fast PWM of Mode 3, $TOP = 255$

For Fast PWM of Mode 7, $TOP = OCRA$

where, D is the Duty cycle that range from 0% to 100%. This duty cycle value is the count value that has to be loaded into the OCR0A (or OCR0B) register.

Example:

A PWM signal is to have 75% duty cycle in Fast PWM of Mode 3. Compute the value for OCR0A.

$$OCR0A = \frac{256 \times 75}{100} - 1 = 192 - 1 = 191$$

Fast PWM Mode: Output Frequency

- **Inverting Fast PWM Duty Cycle**

The formula for frequency computation is the same as for non-inverting fast PWM. But the duty cycle of the **Inverting mode** Fast PWM signal is calculated using the following formula.

$$D = \left(1 - \frac{OCR0x}{TOP+1}\right) \cdot 100\% ; x = A \text{ or } B$$

For Fast PWM of Mode 3, $TOP = 255$
For Fast PWM of Mode 7, $TOP = OCRA$

where, D is the Duty cycle that range from 0% to 100%. This duty cycle value is the count value that has to be loaded into the OCR0A (or OCR0B) register.

- **Example:** A PWM signal is to have 75% duty cycle in Fast PWM of Mode 3. Compute the value for OCR0A.

$$OCR0A = 255 - \frac{256 \times 75}{100} = 255 - 192 = 63$$

Phase correct PWM Mode

- ❖ The phase correct PWM mode (**WGM02:0 = 1 or 5**) provides a high-resolution phase correct PWM waveform generation option. The phase correct PWM mode is based on a **dual-slope operation**. The counter counts repeatedly from BOTTOM to TOP and then from TOP to BOTTOM.
- ❖ In non-inverting Compare Output mode, the Output Compare (OC0x) is cleared on the compare match between TCNT0 and OCR0x while up counting, and set on the compare match while down counting.
- ❖ In inverting Output Compare mode, the operation is inverted.
- ❖ The dual-slope operation has a lower maximum operation frequency than the single-slope operation.
- ❖ However, due to the symmetric feature of the dual-slope PWM modes, these modes are preferred for motor control applications.

Phase correct PWM Mode

- ☐ In phase correct PWM mode, the counter is incremented until the counter value matches TOP. **When the counter reaches TOP, it changes the count direction.**
- ☐ The TCNT0 value will be equal to TOP for one timer clock cycle. The timing diagram for the phase correct PWM mode is shown on **Figure 12.7.**
- ☐ The TCNT0 value is in the timing diagram shown as a histogram for illustrating the **dual-slope operation.**
- ☐ The diagram includes non-inverted and inverted PWM outputs.
- ☐ The small horizontal line marks on the TCNT0 slopes represent compare matches between OCR0x and TCNT0.

Phase correct PWM Mode

- The diagram includes non-inverted and inverted PWM outputs.
- The **small horizontal line marks** on the TCNT0 slopes represent **compare matches between OCR0x and TCNT0**.
- The **Timer/Counter Overflow Flag (TOV0)** is set each time the counter reaches **BOTTOM**.
- The **Interrupt Flag** can be used to **generate an interrupt** each time the **counter reaches the BOTTOM** value.

Bit	7	6	5	4	3	2	1	0	
0x15 (0x35)	-	-	-	-	-	OCF0B	OCF0A	TOV0	TIFR0
Read/Write	R	R	R	R	R	RW	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

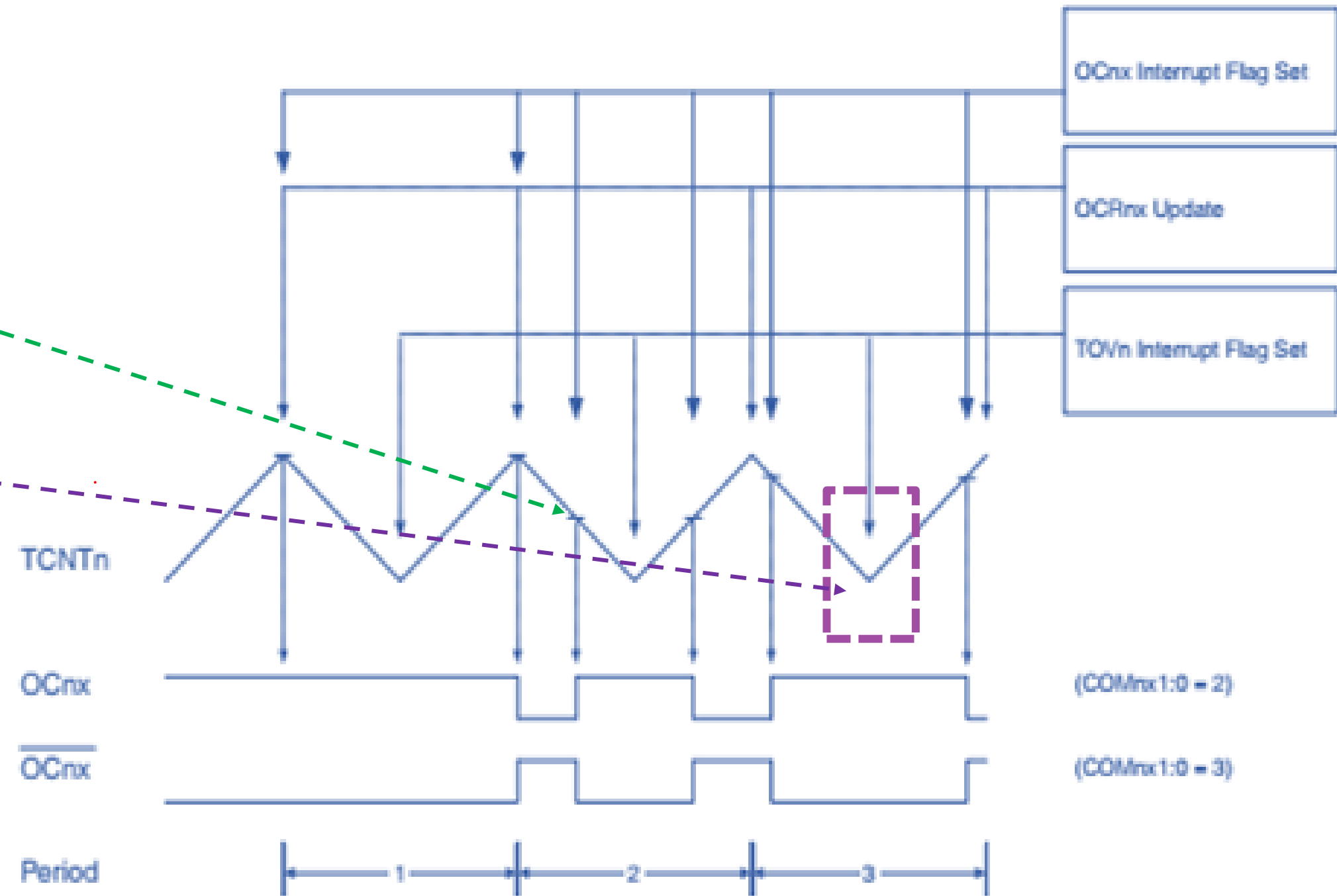


Figure 12.7 Timing Diagram of the Phase Correct PWM Mode

Phase Correct PWM Mode

In Phase Correct PWM mode, the compare unit allows the generation of PWM waveforms on the OC0x pins. **Table 12.7** shows the COM0B1:0 bit functionality when the WGM02:0 bits are set to Phase Correct PWM mode.

Table 12-7. Compare Output Mode, Phase Correct PWM Mode⁽¹⁾

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	<u>Clear OC0B on Compare Match when up-counting. Set OC0B on Compare Match when down-counting.</u>
1	1	<u>Set OC0B on Compare Match when up-counting. Clear OC0B on Compare Match when down-counting.</u>

Phase Correct PWM Mode

- ❖ In phase correct PWM mode, the compare unit allows the generation of PWM waveforms on the OC0x pins. Setting the COM0x1:0 bits to **10** will produce a **non-inverted PWM**.
- ❖ An **inverted** PWM output can be generated by setting the COM0x1:0 to **11**.
- ❖ Setting the COM0A0 bits to **01** allows the **OC0A pin** to toggle on Compare Matches if the WGM02 bit is set. This option is not available for the OC0B pin (**Table 12-7**).

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Phase correct PWM Mode

- The PWM waveform is generated by clearing (or setting) the OC0x Register at the compare match between OCR0x and TCNT0 when the counter increments, and setting (or clearing) the OC0x Register at compare match between OCR0x and TCNT0 when the counter decrements.
- The PWM frequency for the output when using phase correct PWM can be calculated by the following equation:

$$f_{PC_PWM} = \frac{f_{clk_IO}}{2 \times N \times TOP}$$

For Phase Correct PWM of Mode 1, $TOP = 255$
For Fast PWM of Mode 5, $TOP = OCRA$

The N variable represents the pre-scale factor (1, 8, 64, 256, or 1024).

Phase Correct PWM Mode: Duty Cycle

- **Non-Inverting Phase Correct PWM Duty Cycle**

The duty cycle of the **Non-Inverting mode for Phase Correct** PWM signal is calculated using the following formula.

$$D = \frac{OCR0x}{TOP} \cdot 100\% ; x = A \text{ or } B$$

For Fast PWM of Mode 3, $TOP = 255$
For Fast PWM of Mode 7, $TOP = OCRA$

where, D is the Duty cycle that range from 0% to 100%. This duty cycle value is the count value that has to be loaded into the OCR0A (or OCR0B) register.

- **Inverting Phase Correct PWM Duty Cycle**

The duty cycle of the **Inverting mode for Phase Correct** PWM signal is calculated using the following formula.

$$D = \left(1 - \frac{OCR0x}{TOP}\right) \cdot 100\% ; x = A \text{ or } B$$

For Fast PWM of Mode 3, $TOP = 255$
For Fast PWM of Mode 7, $TOP = OCRA$

where, D is the Duty cycle that range from 0% to 100%. This duty cycle value is the count value that has to be loaded into the OCR0A (or OCR0B) register.

Phase correct PWM Mode

- The **extreme values for the OCR0A Register represent special cases** when generating a PWM waveform output in the phase correct PWM mode.
- If the **OCR0A is set equal to BOTTOM**, the output will be continuously low and if set equal to MAX the output will be continuously high for non-inverted PWM mode.
- For inverted PWM, the output will have the opposite logic values.

Example to Self-Practice

Calculate the PWM frequency for the output when using Fast PWM Mode (Mode 3) and Phase Correct PWM Mode (Mode 1) when f_{clk_IO} is 10 MHz and the pre-scale factors are 1, 8, 64, 256, or 1024.

Solution:

The PWM frequency for the **Fast PWM Mode**:

$$f_{FAST_PWM} = \frac{f_{clk_IO}}{N \times 256} = \frac{10}{1024 \times 256} = 38.15 \text{ Hz}$$

The PWM frequency for the **Phase Correct PWM Mode**:

$$f_{PC_PWM} = \frac{f_{clk_IO}}{N \times 510} = \frac{10}{1024 \times 510} = 19.15 \text{ Hz}$$

Important!

Note that in those WGM modes where OCR0A sets the frequency (modes 2, 5, and 7) then you can't do PWM on channel A (because OCR0A couldn't be used for two things at once, i.e., **setting both PWM frequency and duty cycle!**).

So, the **only active PWM channel** in those modes is **channel B** and you set the duty cycle of that (as always) **with OCR0B** (set to some value between 0 and the OCR0A upper limit). As such, the PWM output will be on **pin 11 (PD5)**.



The PWM output for **channel B** with **OCR0B** will be on **pin 11 (PD5)**.

Example for Practice

Use ATmega328p **Timer 0 Fast PWM mode with TOP** at OCR0A (mode 7). With simulation, the frequency changes with OCR0A value loaded. Calculate the frequencies with OCR0A = 100 and with OCR0A = 200. Calculate the duty cycles in this mode. The Pre-scaler value, $N = 1$, and the system clock frequency, $f_{clk_IO} = 16$ MHz.

The PWM frequency for the **Fast PWM Mode**:

$$f_{PWM} = \frac{f_{clk_IO}}{N \times (1 + OCR0A)} = \frac{16}{1 \times 101} = 148.4 \text{ kHz}$$

$$f_{PWM} = \frac{f_{clk_IO}}{N \times (1 + OCR0A)} = \frac{16}{1 \times 201} = 79.6 \text{ kHz}$$

Considering non-inverting mode and OCR0B = 50,

(i) Duty cycle = $OCR0B / OCR0A = 50 / 100 = 0.50$

(ii) Duty cycle = $50 / 200 = 0.25$

```

#ifndef F_CPU
#define F_CPU 8000000UL    // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
    // OC0B pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
    DDRD |= (1<<PD5);
    OCR0B= 191; // Load 191 into OCR0B for setting its duty cycle to 75%

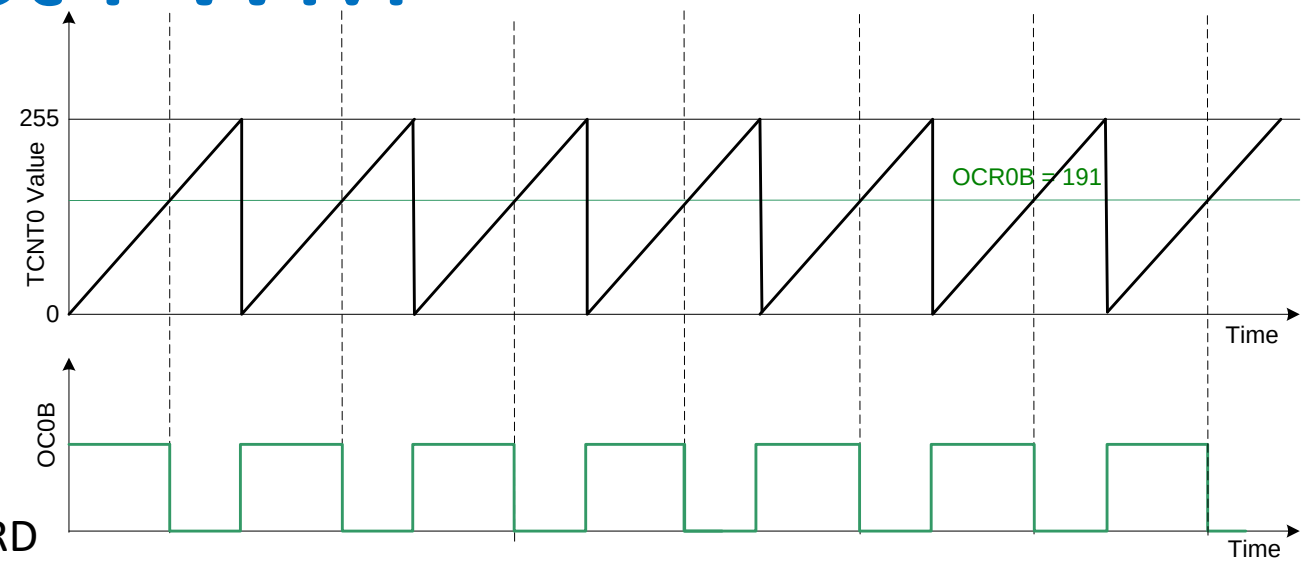
```

```

    // Configure TCCR0A and TCCR0B register for (i) non-inverting (10), (ii) Fast PWM mode 3 (011), (iii) No
    // Pre-scalar (001) for frequency control. By default, 0s are there, but you may set them explicitly.
    TCCR0A |= (1 << COM0B1) | (1<<WGM01) | (1<<WGM00);
    TCCR0B |= (1<<CS00);

    while(1);
    return 0;
}

```



Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

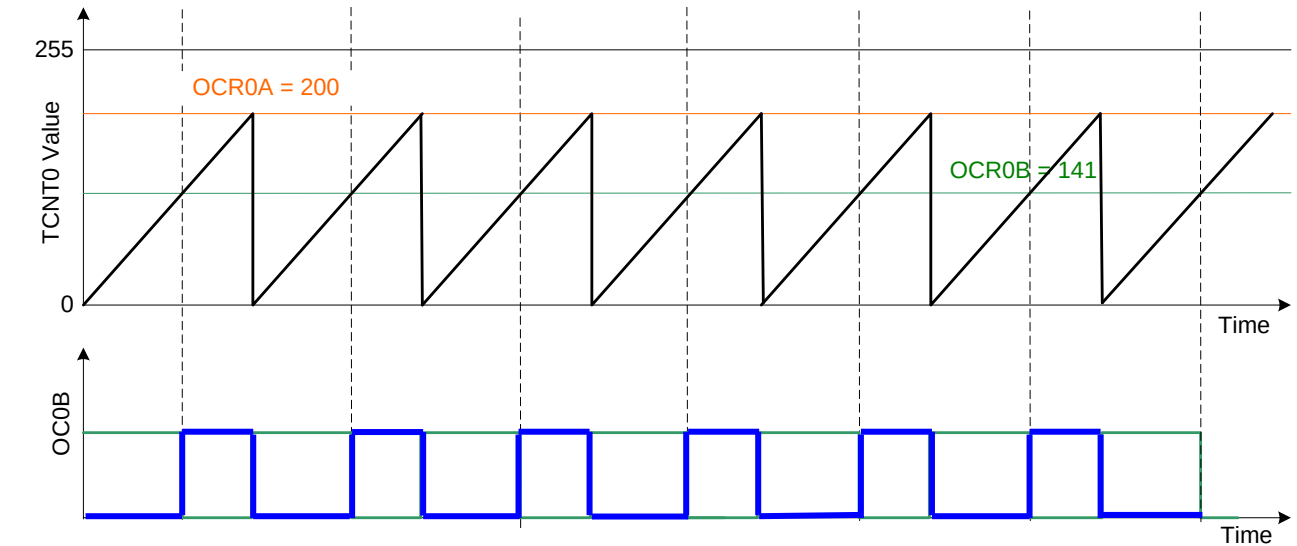
Programming Arduino for Fast PWM

```
#ifndef F_CPU
#define F_CPU 8000000UL // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
    // OCB pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
    DDRD |= (1<<PD5);
    OCR0A = 200; // Top Value of 200 (must be equal or greater than the Duty Cycle)
    OCR0B = 141; // Load 191 into OCR0B for setting its duty cycle to 75%

    // Configure TCCR0A and TCCR0B register for (i) inverting (11), (ii) Fast PWM mode 7 (111), (iii) No
    // Pre-scalar (001) for frequency control. By default, 0s are there, but you may set them explicitly.
    TCCR0A |= (1<<COM0B1) | (1<<COM0B0) | (1<<WGM01) | (1<<WGM00);
    TCCR0B |= (1<<WGM02) | (1<<CS0 0);
    // TCCR0A=0b00110011; TCCR0B=0b00001001

    while(1);
    return 0;
}
```



Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

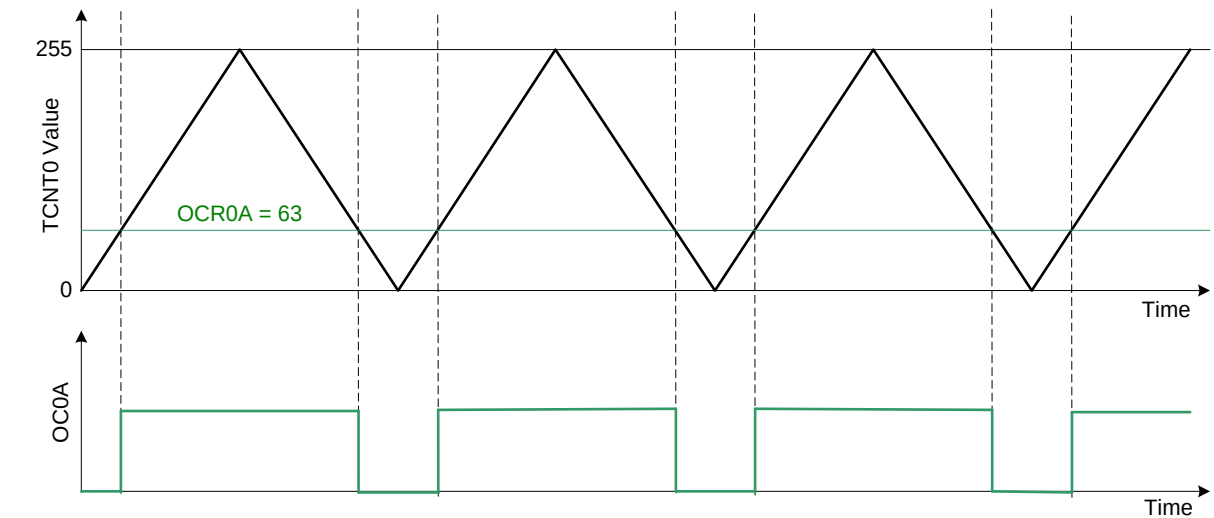
Programming Arduino for Phase-Correct PWM

```
#ifndef F_CPU
#define F_CPU 8000000UL // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
    // OCOA pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
    DDRD |= (1<<PD6);
    OCR0A= 63; // Load 63 into OCR0A for setting its duty cycle to 75%

    // Configure TCCR0A and TCCR0B register for (i) inverting (11), (ii) Phase correct PWM mode 1 (001),
    // (iii) No Pre-scalar (001) for frequency control. By default, 0s are there, but you may set them explicitly.
    TCCR0A |= (1 << COM0A1) | (1 << COM0A0) | (1<<WGM00);
    TCCR0B |= (1<<CS00);

    while(1);
    return 0;
}
```



Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

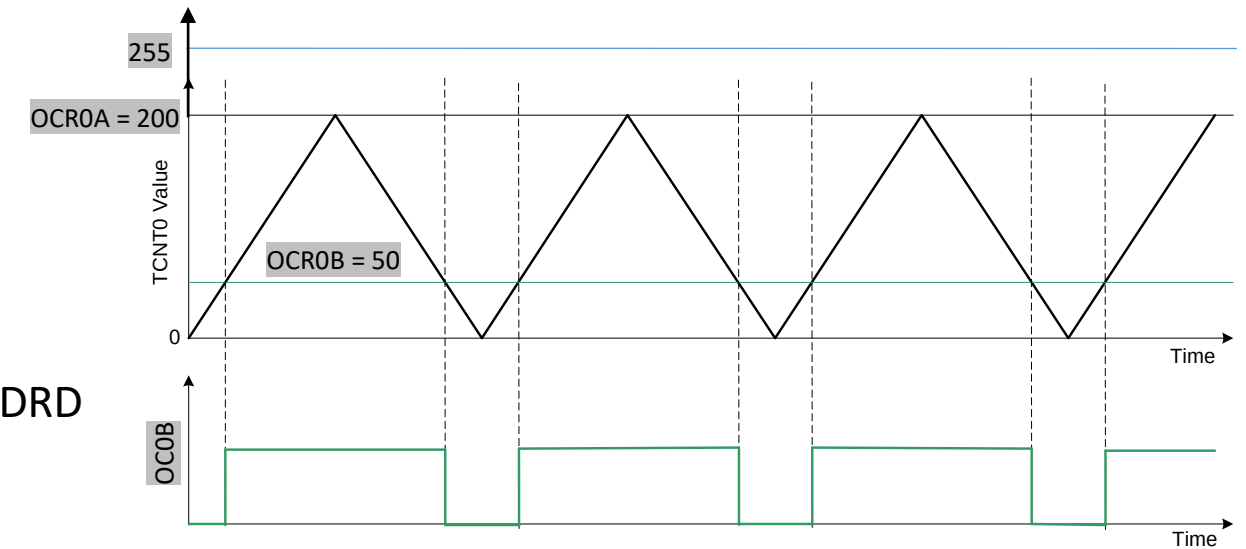
Programming Arduino for Fast PWM

```
#ifndef F_CPU
#define F_CPU 8000000UL // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
    // OC0B pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
    DDRD |= (1<<PD5);
    OCR0A = 200; // Top Value of 200 (must be equal or greater than the Duty Cycle)
    OCR0B = 50; // Load 50 into OCR0B for setting its duty cycle to 75%

    // Configure TCCR0A and TCCR0B register for (i) inverting (11), (ii) Phase Correct PWM mode 5 (101), (iii) No
    // Pre-scalar (001) for frequency control. By default, 0s are there, but you may set them explicitly.
    TCCR0A |= (1<<COM0B1) | (1<<COM0B0) | (1<<WGM00);
    TCCR0B |= (1<<WGM02) | (1<<CS00);
    // TCCR0A=0b00110001; TCCR0B=0b00001001

    while(1);
    return 0;
}
```



Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	